

AI-Native Software Design Factory (SDF)

Reference Architecture, Operating Model & Implementation Blueprint

Design baseline tổng hợp từ quá trình phân tích

19/09/2026

Mục lục

1. Executive Summary
2. Mục tiêu và Non-goals
3. Nguyên tắc thiết kế
4. Lean + PDCA là operating philosophy
5. End-to-end lifecycle
6. Traceability: từ design đến implementation task
7. Knowledge architecture: Intent, Reality và Evolution
8. Change model và lifecycle của artifacts
9. Agent architecture
10. Agent governance và trust boundaries
11. Agent Runtime abstraction
12. VS Code + Codex + extensions
13. Repository reference layout
14. Architecture fitness functions
15. Technical Debt, incidents và lessons
16. Runtime evidence và design drift
17. Security cho Software Design Factory
18. Data classification và context policy
19. Factory Reliability & Disaster Recovery
20. Cost & resource governance
21. Scrum có cần thiết không?
22. Definition of Ready và Definition of Done
23. Metrics và Factory evaluation
24. Agent evaluation harness
25. Golden paths và escape hatches
26. Maturity model
27. Lộ trình triển khai
28. Minimum Viable SDF
29. Acceptance criteria cho SDF v1
30. Decision log ban đầu
31. Kiến trúc mục tiêu tóm tắt
32. Kết luận
33. Phụ lục và nguồn tham khảo

Thông tin tài liệu

Thuộc tính	Giá trị
Tên	AI-Native Software Design Factory (SDF)
Phiên bản	1.0 - Proposed Build Baseline
Ngày	19/09/2026
Mục tiêu	Định nghĩa kiến trúc tham chiếu, operating model, governance và lộ trình triển khai SDF có AI Agent
Đối tượng	Software Architect, Tech Lead, Developer, Platform Engineer, Security, SRE, Product/Engineering Manager
Phạm vi	Từ idea thô đến design, implementation, runtime evidence và continuous evolution
Trạng thái	Sẵn sàng dùng làm baseline trước khi bắt tay xây dựng

Quy ước chuẩn tắc

Trong tài liệu này:

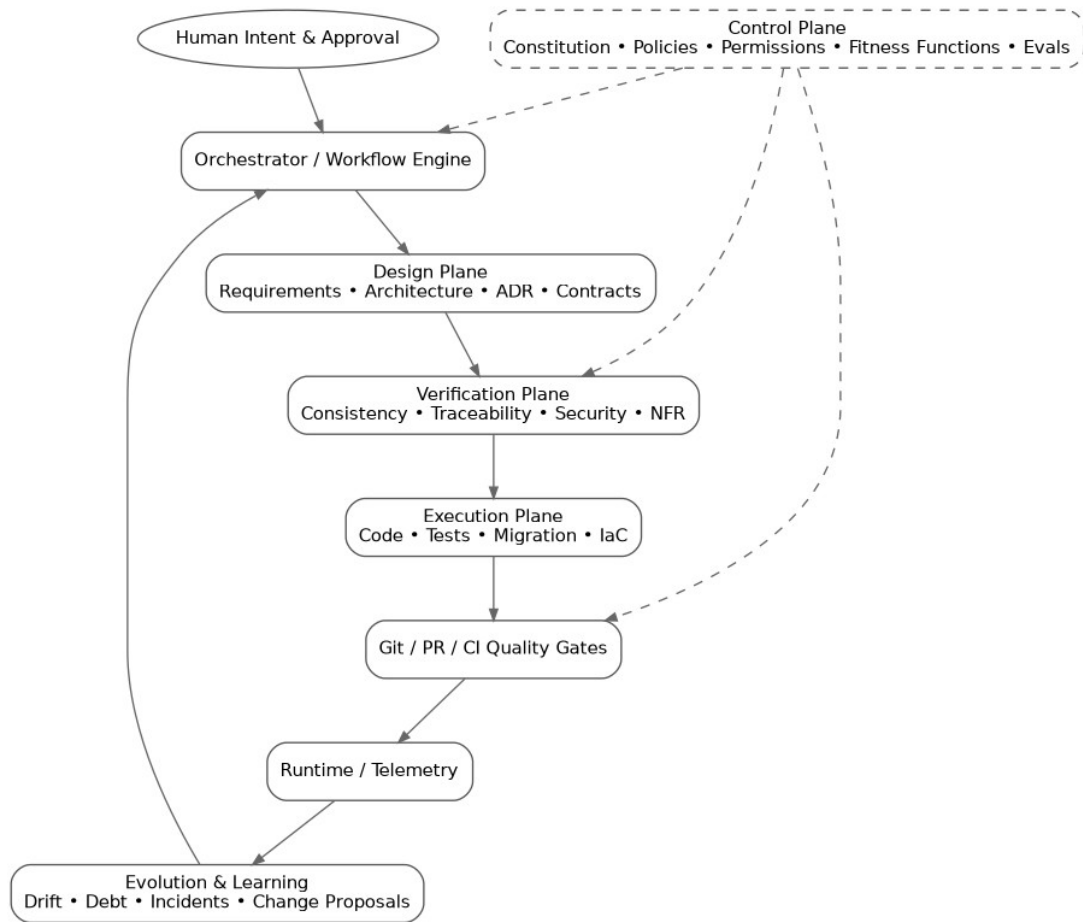
- **MUST**: bắt buộc để duy trì tính đúng đắn hoặc khả năng quản trị của Factory.
- **SHOULD**: mặc định nên thực hiện; chỉ bỏ qua khi có lý do rõ ràng.
- **MAY**: tùy chọn theo bối cảnh.

1 1. Executive Summary

AI-Native Software Design Factory không phải một hệ thống sinh tài liệu tự động. Nó là một **living engineering system** quản lý sự tiến hóa của engineering intent qua thời gian, từ problem và requirement cho tới architecture, decision, task triển khai, code, test, deployment, runtime evidence và learning.

Factory được xây trên bốn nguyên lý nền tảng:

1. **Automation must never become the source of truth.** AI, graph, IDE và index chỉ tạo hoặc dẫn xuất thông tin; canonical artifacts có version mới là authority.
2. **Everything derived must be reproducible.** Graph, diagram, report, tài liệu xuất bản và agent configuration dẫn xuất phải có thể rebuild từ canonical sources.
3. **Every irreversible decision requires explicit human accountability.** AI có thể phân tích, đề xuất, kiểm chứng và thực thi; con người giữ thẩm quyền đối với business intent, major architecture trade-off, risk acceptance, security exception và irreversible migration.
4. **Every material change must be traceable.** Không có implementation change quan trọng nào được tồn tại như một “orphan change” không truy được về intent/design upstream và evidence downstream.



Kiến trúc tổng thể của AI-Native SDF

Nền tảng triển khai ngắn hạn được khuyến nghị là **VS Code + Codex + Git + structured artifacts + MCP + Graphify + CI**. Tuy nhiên, kiến trúc dài hạn phải là **Codex-first, not Codex-dependent**: Codex chỉ là một implementation của Agent Runtime, còn truth, workflow, policies, tools, schemas và evaluation suites phải độc lập với AI provider.

2. Mục tiêu và Non-goals

2.1 Mục tiêu

SDF MUST:

- biến idea thô thành design increment có thể triển khai và kiểm chứng;
- giảm thời gian từ discovery đến implementation mà không đánh đổi traceability;
- duy trì knowledge xuyên suốt dự án dài hạn và thay đổi nhân sự;
- tự động phát hiện inconsistency, design drift, broken trace và architecture violations;
- đưa runtime evidence quay trở lại design process;
- hỗ trợ nhiều AI Agent nhưng giữ human authority ở các quyết định quan trọng;
- cho phép thay Agent Runtime/AI provider mà không rewrite canonical knowledge;
- kiểm soát security, cost, permissions, quality và auditability của AI Agent;
- tiến hóa liên tục bằng Lean + PDCA.

2.2 2.2 Non-goals

SDF không nhằm:

- thay thế hoàn toàn Software Architect, Product Owner, Developer hoặc Security/SRE;
- biến mọi hoạt động engineering thành tài liệu;
- yêu cầu tất cả dự án dùng Scrum;
- tạo một universal multi-agent platform trước khi nhu cầu thực tế được chứng minh;
- coi Knowledge Graph là source of truth duy nhất;
- cho AI tự phê duyệt major risk hoặc policy mà không có accountable human.

3 3. Nguyên tắc thiết kế

3.1 3.1 Canonical truth trước automation

Canonical sources SHOULD dùng các định dạng có thể đọc bởi người và máy:

- Markdown cho problem, requirement, design note và ADR;
- YAML/JSON cho structured spec, traceability và policies;
- OpenAPI/AsyncAPI/JSON Schema cho executable contracts;
- source code, tests, IaC và migrations trong Git;
- telemetry schemas cho runtime evidence.

PDF/DOCX/HTML được xem là **publication artifacts**, không phải canonical source mặc định.

3.2 3.2 Model-first, document-second

Architecture SHOULD tồn tại như structured model trước khi render thành diagram hoặc prose. Một model có thể sinh nhiều views; một diagram không nên là nơi duy nhất chứa architectural truth.

3.3 3.3 Small batch và progressive elaboration

Design được thực hiện theo **design increment**, không phải cố hoàn thiện toàn bộ hệ thống trước khi build. Mỗi increment MUST giảm uncertainty hoặc tạo ra một decision có thể kiểm chứng.

3.4 3.4 Build quality in

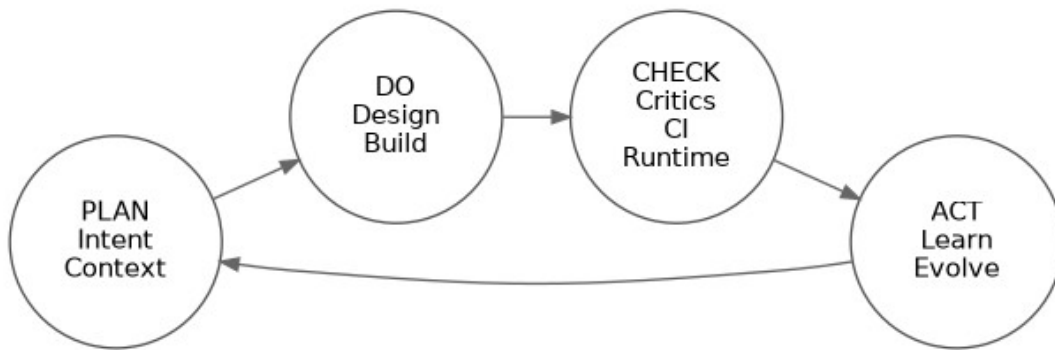
Consistency, traceability, security, NFR và policy validation MUST chạy trong flow, không chờ “review cuối phase”.

3.5 3.5 Human authority, machine execution

Human tập trung vào intent, trade-off, exception và accountability. AI tập trung vào analysis, synthesis, generation, validation, impact analysis, implementation và repetitive checks.

4 4. Lean + PDCA là operating philosophy

Lean là nguyên tắc tối ưu flow; PDCA là vòng học và kiểm soát.



PDCA trong SDF

4.1 4.1 Lean rules

Factory SHOULD áp dụng:

- **Value:** mỗi artifact phải phục vụ một quyết định, implementation hoặc validation.
- **Eliminate waste:** tránh prose dư thừa, diagram không sử dụng và dữ liệu duplicate.
- **Small batch:** thiết kế theo capability/change nhỏ.
- **Pull:** chỉ elaborate chi tiết khi downstream cần.
- **Limit WIP:** giới hạn số design topic đang mở.
- **Fast feedback:** review sớm, runtime feedback nhanh.
- **Build quality in:** check tự động trước merge.
- **Continuous improvement:** lỗi lặp lại phải biến thành rule/check/eval.

Một rule Lean quan trọng:

Không tạo artifact nếu chưa xác định được ai dùng nó và quyết định/validation nào phụ thuộc vào nó.

4.2 4.2 Ba tầng PDCA

Tầng	Plan	Do	Check	Act
Feature/Change	scope, requirement, risk	design + implementation	critic + test + evidence	fix + rule
Product/System	architecture roadmap	evolve system	drift, debt, SLO, incidents	refactor/migrate
Factory	process/policy	run SDF	lead time, defects, cost, overrides	cải tiến workflow/agents

5 5. End-to-end lifecycle

Flow chuẩn:

Idea → Frame → Specify → Model → Decide → Design → Verify → Approve → Implement → Observe → Learn → Evolve

5.1 5.1 Stage model

Stage	Mục tiêu	Output chính	Human gate
Constitution	xác lập invariant	principles, policies, standards	Architect/Engineering leadership
Idea Capture	hiểu hypothesis	Idea Brief, assumptions	Product owner
Evidence & Framing	hiểu problem	problem, users, evidence, scope	Product owner
Specification	tạo testable intent	FR/NFR, acceptance criteria	PO + Architect
System Modeling	mô hình hệ thống	domain + architecture model	Architect
Decision	giải trade-off	options + ADR	Architect/owner
Detailed Design	đủ để build	APIs, data, security, ops	Tech leads khi cần
Verification	tìm inconsistency/risk	findings	automated + human
Approval	chấp thuận baseline	approved change/design increment	accountable owner
Implementation	hiện thực hóa	code/test/IaC/migration	PR review
Observe	thu evidence	telemetry, SLO, incidents	continuous
Evolution	học và thay đổi	drift/debt/change proposals	continuous/periodic

6 6. Traceability: nguyên tắc bắt buộc từ design đến từng implementation task

6.1 6.1 Câu trả lời chuẩn

Đúng, mọi implementation task có tác động vật chất tới software MUST truy vết được tới design intent liên quan. Tuy nhiên không có nghĩa “mỗi tài liệu thiết kế phải map 1:1 với một task”. Quan hệ thực tế là **many-to-many và hai chiều**.

Một requirement có thể sinh nhiều design components và nhiều task. Một task có thể thực hiện đồng thời nhiều requirement, ADR hoặc contract. Điều Factory phải bảo đảm là **không có material task hoặc material change bị orphan**.



Traceability chain chuẩn

Trace chain chuẩn:

Problem/Outcome → Requirement/NFR → Design → ADR/Contract/Threat → Dev Task → Code/Config/IaC → Test/Verification → Runtime Evidence

6.2 6.2 Traceability levels

Không nên áp full bureaucracy cho mọi task. SDF dùng risk-based traceability:

Level	Loại task	Trace tối thiểu
T0 - Housekeeping	formatting, typo, dev tooling không đổi behavior	Task ↔ PR/commit; lý do ngắn
T1 - Standard Implementation	feature/bugfix đổi behavior	Requirement/bug intent ↔ design/component ↔ task ↔ code ↔ test
T2 - Critical Change	architecture, API, data, security, reliability, migration, compliance	Full chain: intent ↔ NFR ↔ design ↔ ADR/contract/threat ↔ task ↔ code/IaC ↔ tests ↔ rollout/runtime evidence

Nếu một “housekeeping task” thực tế thay đổi runtime behavior, nó tự động được nâng ít nhất lên T1.

6.3 6.3 Bidirectional traceability

Factory MUST hỗ trợ hai câu hỏi:

Forward trace:

FR-023 đã được implement ở đâu, bởi task nào, test nào và runtime evidence nào chứng minh nó hoạt động?

Backward trace:

Dòng code/config này được thêm vì requirement/decision nào và ai đã phê duyệt decision đó?

6.4 6.4 Traceability quality gates

CI SHOULD fail hoặc raise blocking finding khi:

- T1/T2 task không có upstream intent;
- changed API/contract không có design/contract update;
- architecture-sensitive code change không có component/ADR linkage khi policy yêu cầu;
- requirement được đánh dấu Implemented nhưng không có task/test;
- deprecated component vẫn có new dependency mà không có exception;
- critical design artifact không có owner/status/version;
- task closed nhưng verification evidence bị thiếu.

6.5 6.5 Không biến traceability thành nhập liệu tay

Developer không nên phải cập nhật 6 file chỉ để nối task với requirement. Agent/CI SHOULD tự đề xuất hoặc tự cập nhật derived links từ PR diff, code ownership, graph và task metadata; human chỉ xác nhận những liên kết có uncertainty cao.

Canonical explicit links nên đủ nhỏ và ổn định, ví dụ:

```
work_item: DEV-142
implements:
  - FR-023
  - NFR-011
affects:
  - CMP-SEARCH
governed_by:
  - ADR-012
verifies_with:
  - TEST-E2E-27
```

7 7. Knowledge architecture: Intent, Reality và Evolution

SDF cần ba graph logic, không nhất thiết ba database vật lý.

7.1 7.1 Intent Graph

Human-owned/canonical, biểu diễn “hệ thống được kỳ vọng là gì”:

- Problem/Outcome
- Requirement/NFR
- Domain concept
- Architecture component
- ADR
- API/Event/Data contract
- Threat/Mitigation
- SLO/Operational intent
- Test intent

7.2 7.2 Reality Graph

Tool-derived, biểu diễn “hệ thống thực tế là gì”:

- repository/module/class/function;
- imports/calls/dependencies;
- DB/schema/migrations;
- deployed infrastructure;
- runtime service/dependency topology;
- telemetry observations.

Graphify phù hợp làm một thành phần của Reality/Context Graph vì dự án hiện cung cấp deterministic AST extraction cho code, provenance cho quan hệ EXTRACTED/INFERRED, query/path operations và MCP access. Nhưng Graphify MUST NOT trở thành sole canonical authority cho design intent.

7.3 7.3 Evolution Graph

Biểu diễn “hệ thống đã thay đổi vì sao”:

- Change Proposal
- migration
- deprecation/replacement
- incident
- technical debt

- exception/waiver
- lesson learned
- superseded ADR

7.4 7.4 Temporal semantics

Long-lived project cần version/time trong graph:

- status
- valid_from
- valid_until
- superseded_by
- introduced_by
- retired_by

Agent phải có khả năng phân biệt decision hiện tại với historical decision.

8 8. Change model và lifecycle của artifacts

8.1 8.1 Change Proposal là first-class object

Mọi thay đổi đáng kể SHOULD bắt đầu bằng một change object:

```
change: CP-027
reason: "Payment latency exceeds target"
affects:
  - NFR-009
  - CMP-PAYMENT
  - ADR-014
risk: high
migration_required: true
```

Flow:

Signal → Change Proposal → Impact Analysis → Decision → Design Delta → Tasks → Implementation → Validation → Evidence

8.2 8.2 Artifact lifecycle

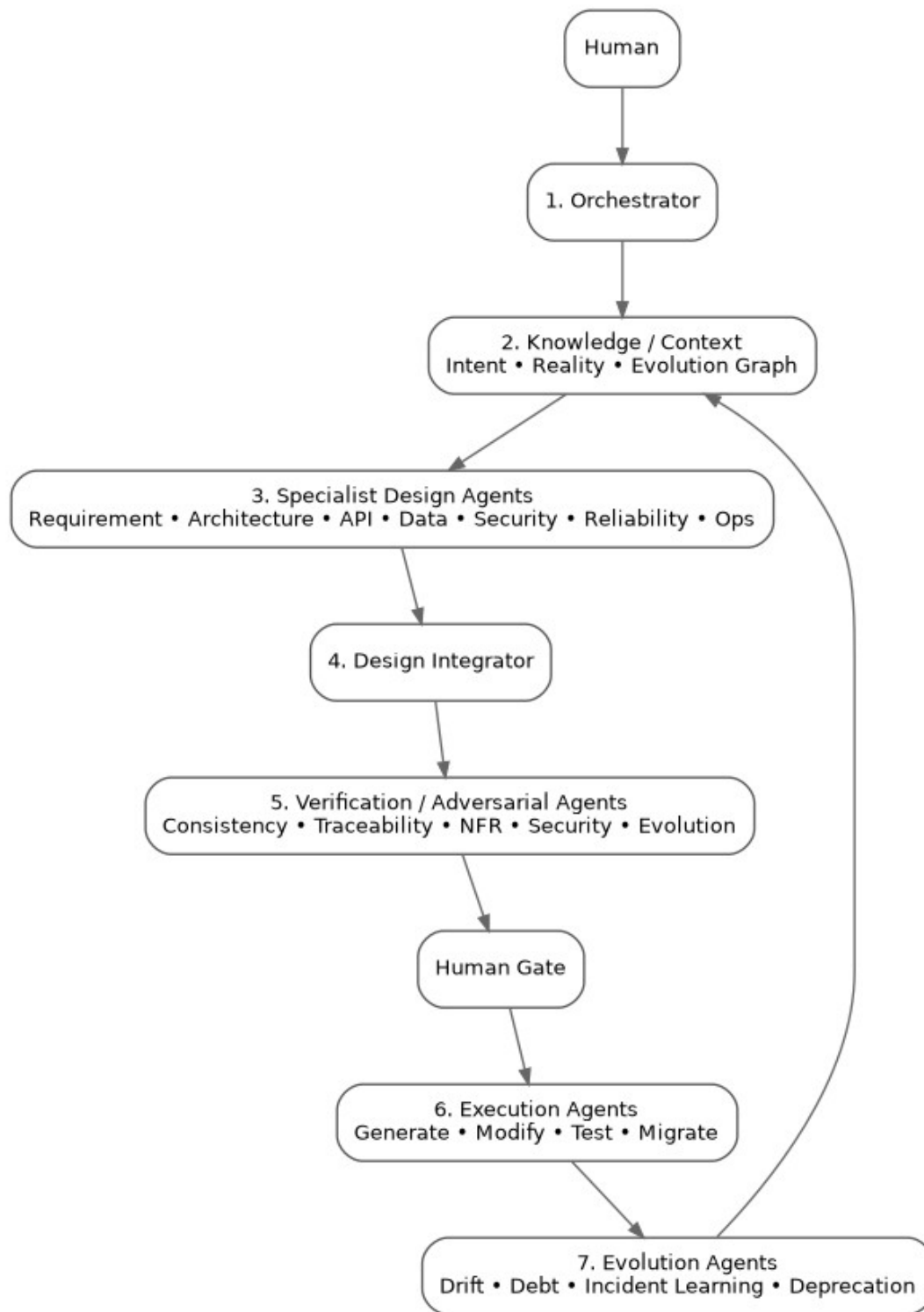
Ví dụ trạng thái:

- Requirement: Proposed → Approved → Implemented → Verified → Deprecated
- ADR: Draft → Proposed → Accepted → Active → Superseded → Retired
- Component: Planned → Active → Legacy → Replacing → Retired
- API: Experimental → Stable → Deprecated → Removed

Status MUST là structured data đủ để Agent và CI hiểu.

9 9. Agent architecture

SDF sử dụng logical layers, không bắt buộc mỗi layer là một service/model riêng.



Các lớp AI Agent

9.1 9.1 Layer 1 - Orchestrator

Trách nhiệm:

- hiểu request/change;
- phân loại risk/trace level;

- query context;
- lập Design Work Item;
- dispatch specialist agents;
- collect/normalize outputs;
- gọi verification;
- đưa decision cần human lên gate.

Orchestrator **MUST NOT** tự âm thầm đổi requirement hoặc waive finding quan trọng.

9.2 9.2 Layer 2 - Knowledge/Context Agents

Nhiệm vụ là tạo **Context Pack** có relevance cao từ:

- Intent Graph;
- Reality Graph;
- Evolution Graph;
- Git history;
- current artifacts;
- relevant incidents/debt.

Mục tiêu là tránh context pollution và không bắt mỗi Agent đọc toàn repo.

9.3 9.3 Layer 3 - Specialist Design Agents

Kích hoạt theo nhu cầu:

- Requirement/Domain
- Architecture
- API/Integration
- Data
- Security/Threat
- Reliability
- Deployment/Operations

Các Agent **MAY** chạy song song sau khi scope/requirements đủ ổn định.

9.4 9.4 Layer 4 - Design Integrator

Integrator hợp nhất outputs và tạo conflict object thay vì tự chọn im lặng khi specialist outputs mâu thuẫn.

9.5 9.5 Layer 5 - Verification/Adversarial Agents

Nên có các critic độc lập:

- Consistency Critic
- Traceability Critic
- NFR Critic
- Security Critic
- Evolution/Compatibility Critic

Critic **SHOULD** read-only đối với canonical design trong review pass.

9.6 9.6 Layer 6 - Execution Agents

Sau approval, Agent có thể:

- cập nhật structured design;
- generate OpenAPI/schema/diagram;
- tạo code/test/migration/IaC;
- update traceability;
- chạy test và validation.

9.7 9.7 Layer 7 - Evolution Agents

Sau deployment:

- Drift Agent
- Debt Agent
- Incident Learning Agent
- Dependency Evolution Agent
- Architecture Health Agent
- Deprecation Agent

10 10. Agent governance và trust boundaries

Agent MUST có identity, version, permissions và output contract.

Ví dụ:

```
agent:
  id: architecture-reviewer
  version: 4.1
permissions:
  repo: read
  graph: read
  write: false
allowed_actions:
  - create_finding
forbidden_actions:
  - approve_architecture
  - waive_policy
output_schema: design-finding.v2
```

10.1 10.1 Trust model

Agent	Có thể	Không thể
Research	đọc nguồn, tạo evidence	tự approve business decision
Architecture	propose design/ADR	tự accept major ADR
Security	raise findings	tự waive critical finding
Reviewer	đọc, đánh giá	sửa canonical artifact trong same review pass
Executor	sửa repo sau approval	thay intent ngoài approved scope
Publisher	build docs	thay canonical source

Agent	Có thể	Không thể
Orchestrator	phối hợp	bypass mandatory gate

10.2 10.2 Human gates

Tối thiểu ba gate:

- **Gate A:** problem/scope được chấp nhận;
- **Gate B:** major architecture/ADR/risk được chấp nhận;
- **Gate C:** design release/critical change được chấp nhận.

Không phải mọi T1 task cần manual architecture gate; policy có thể tự động approve low-risk changes khi fitness functions/evals đạt.

11 11. Agent Runtime abstraction: Codex-first, not Codex-dependent

11.1 11.1 Ngăn hạn dùng chung Codex

Các logical Agent có thể dùng chung Codex và khác nhau ở:

- role/instructions;
- context;
- model/reasoning effort;
- allowed tools;
- permissions;
- output schema.

Codex hiện hỗ trợ AGENTS.md theo hierarchy, project-level .codex/config.toml, custom subagent roles và external MCP servers. Điều này đủ thực dụng để bootstrap SDF mà chưa cần custom orchestration platform.

11.2 11.2 Dài hạn: Agent Runtime Interface (ARI)

Factory SHOULD định nghĩa internal runtime contract:

```
run(task, role, context, tools, permissions, expected_schema) -> canonical_result
```

Kiến trúc:

Factory → ARI → Runtime Adapter → Codex / Runtime B / Runtime C

Canonical Agent Spec không nên chỉ tồn tại trong .codex/agents/*.toml. Provider-specific configuration là deployment artifact/adaptor.

11.3 11.3 Năm thứ không được khóa vào Codex

- Knowledge
- Workflow
- Policies
- Tools
- Evaluations

AGENTS.md SHOULD được xem là một Codex-facing compiled instruction layer từ canonical constitution/policies, không phải nơi duy nhất lưu governance.

11.4 11.4 MCP là tool boundary, không phải runtime abstraction

MCP phù hợp cho tools/context integration. ARI là abstraction riêng cho việc chạy Agent. Phân tách này tránh phụ thuộc protocol/tool lifecycle của một vendor.

Lưu ý hiện tại: Codex vẫn kết nối được external MCP servers; standalone Codex MCP server đã bị loại bỏ và OpenAI hướng integration cần conversation/auth/approval sang app-server protocol, vốn đang được ghi là experimental cho production. Vì vậy SDF không nên lấy Codex app-server hay một proprietary endpoint làm control-plane core.

12 12. VS Code + Codex + extensions: chiến lược ngắn hạn và dài hạn

12.1 12.1 Ngắn hạn: phù hợp

0-6 tháng đầu, stack đề xuất:

- VS Code: developer cockpit;
- Codex IDE + CLI: primary agent execution;
- Git: canonical versioning;
- AGENTS.md: Codex-specific project guidance;
- .codex/config.toml: local/project runtime config;
- MCP: tool/context integration;
- Graphify: Reality/Context graph adapter;
- scripts + CI: deterministic checks và quality gates.

Ưu điểm:

- feedback loop ngắn;
- dev thấy spec, code, diff, tests, Agent trong cùng môi trường;
- không cần xây platform trước khi hiểu workflow;
- cùng repo giữ rules, schemas, evals và artifacts.

12.2 12.2 Dài hạn: VS Code chỉ là client

Core logic MUST NOT chỉ tồn tại trong extension.

- Extension dùng cho editor UX: inline diagnostics, graph visualization, code lens, design diff panel.
- MCP/CLI/API dùng cho domain capabilities: query graph, impact analysis, validate architecture, create ADR, traceability, runtime query.

Target architecture:

Factory Control Plane → MCP/API/CLI → VS Code / CI/CD / Web / other IDE / other Agent Runtime

VS Code hiện quảng bá khả năng dùng nhiều agent/model/provider và MCP. Điều này phù hợp với vai trò “replaceable workstation client”, nhưng Factory vẫn phải giữ portability độc lập với editor.

13 13. Repository reference layout

Một layout khởi đầu:

```
software/
├── AGENTS.md
├── .codex/
│   ├── config.toml
│   └── agents/
├── constitution/
│   ├── principles.yaml
│   ├── policies.yaml
│   └── quality-gates.yaml
├── design/
│   ├── requirements/
│   ├── domain/
│   ├── architecture/
│   ├── decisions/
│   ├── contracts/
│   ├── security/
│   └── operations/
├── knowledge/
│   ├── intent.yaml
│   ├── evolution.yaml
│   └── schemas/
├── agents/
│   ├── specs/
│   ├── prompts/
│   ├── workflows/
│   └── evaluations/
├── tools/
│   ├── impact/
│   ├── traceability/
│   ├── architecture-lint/
│   └── drift/
├── graphify-out/           # derived/rebuildable
├── src/
└── tests/
```

Nguyên tắc: graphify-out/, rendered docs, diagrams generated và reports là derived artifacts; canonical source nằm ở design/, constitution/, knowledge/, agents/, code và tests.

14 14. Architecture fitness functions

Principle chỉ có giá trị lâu dài nếu được biến thành executable checks.

Ví dụ:

ARCH-07: Service A MUST NOT connect trực tiếp tới database owned bởi Service B.

SEC-012: Public API MUST có authentication, authorization, rate limit và audit event theo policy.

REL-004: External dependency MUST định nghĩa timeout, retry/backoff, failure behavior và observability.

Fitness functions chạy ở:

- pre-commit/lint khi rê;
- PR/CI khi cần cross-file/repo analysis;
- continuous drift monitoring khi phụ thuộc runtime reality.

15 15. Technical Debt, incidents và lessons là first-class data

15.1 15.1 Technical Debt

Debt object nên có:

- ID
- introduced_by
- affects
- violates
- risk
- “interest”/cost of delay
- owner
- target/review date

Factory theo dõi debt age, concentration, growth và repeated rule violations thay vì chỉ đếm TODO.

15.2 15.2 Incident feedback loop

Incident không kết thúc ở postmortem:

Incident → Lesson → Design/Architecture Rule → Fitness Function → Agent Instruction/Eval

Nếu cùng failure mode lặp lại, Factory phải coi đó là defect của process/control plane, không chỉ defect của team implementation.

16 16. Runtime evidence và design drift

Runtime evidence SHOULD được nối với NFR/design assumptions:

- latency/throughput/error rate;
- SLO/SLA;
- service dependency topology từ tracing;
- resource utilization;
- failure modes;
- security events;
- rollout/rollback evidence.

Drift detection so sánh Intent Graph với Reality Graph:

- dependency mới không có ADR/design update;
- API implementation khác OpenAPI;
- migration khác data model;
- IaC khác deployment model;
- runtime topology khác intended topology.

Drift không luôn là lỗi; nó có thể là signal để cập nhật intent hoặc rollback reality. Quyết định cần explicit resolution.

17 17. Security cho chính Software Design Factory

Factory có quyền mạnh, vì vậy cần threat model riêng.

17.1 17.1 Threats chính

- prompt injection từ source/docs/issues/web content;
- malicious/compromised MCP server hoặc extension;
- secret exfiltration;
- Agent vượt quyền/scope;
- supply-chain poisoning;
- compromised generated code/dependency;
- poisoned Knowledge Graph hoặc inferred edge bị coi như fact;
- log/telemetry làm lộ sensitive context;
- unreviewed autonomous deployment.

17.2 17.2 Security controls

MUST có:

- least privilege per Agent/tool;
- secret isolation;
- allowlist/approval cho high-risk actions;
- provenance cho derived knowledge;
- sandbox khi chạy untrusted code;
- artifact signing/audit trail theo mức cần thiết;
- data classification và context policy;
- dependency/source verification;
- human approval cho critical exception/deploy/migration.

18 18. Data classification và context policy

Classification gợi ý:

- Public
- Internal
- Confidential
- Restricted

Policy quyết định:

- Agent/runtime nào được nhận class nào;
- dữ liệu nào được đưa ra external provider;
- retention/logging;
- remote MCP eligibility;
- redaction/tokenization;
- quyền export artifacts.

Context Pack MUST respect classification trước relevance.

19 19. Factory Reliability & Disaster Recovery

Factory không được trở thành single point of failure của engineering.

19.1 19.1 Recovery principle

Nếu AI provider, graph service hoặc extension ngừng hoạt động, team vẫn phải có khả năng:

- đọc canonical specs;
- build/test code;
- review/merge bằng degraded manual flow;
- rebuild derived graph/index/docs khi service phục hồi.

19.2 19.2 Rebuildability

MUST có runbook/script để rebuild:

- Intent/Reality indexes từ canonical sources;
- Graphify output;
- rendered diagrams/docs;
- traceability reports;
- agent-facing compiled instructions.

Critical canonical data MUST không phụ thuộc ephemeral AI session memory.

20 20. Cost & resource governance

Multi-agent có thể tăng chất lượng nhưng cũng tăng token/compute. Factory SHOULD có:

- workflow budget;
- concurrency limits;
- model tiering;
- escalation rules;
- deterministic-tool-first policy;
- cost attribution theo project/workflow;
- caching/reuse khi safe;
- policy tránh chạy critic nặng trên trivial change.

Nguyên tắc:

Dùng deterministic computation cho điều có thể xác định; dùng AI cho ambiguity, synthesis và judgment.

21 21. Scrum có cần thiết không?

Không. Scrum là một delivery cadence, không phải operating system của SDF.

Nền mặc định được đề xuất:

Lean + PDCA + Continuous Flow/Kanban

Scrum MAY được dùng cho team có stable backlog, incremental product delivery và stakeholder feedback theo sprint. Platform/SRE/architecture/research-heavy work thường phù hợp hơn với continuous flow.

Dù không dùng Scrum, vẫn cần cadence:

- Continuous: CI, security, traceability, drift checks;
- Weekly: design/risk/debt review;
- Monthly/Quarterly: architecture health, roadmap, policy/constitution review.

22 22. Definition of Ready và Definition of Done

22.1 22.1 Design Ready

Một change được xem là ready khi:

- problem/outcome rõ;
- owner rõ;
- scope đủ rõ;
- critical unknowns được ghi nhận;
- required evidence có hoặc có plan lấy evidence;
- trace level/risk được phân loại.

22.2 22.2 Design Done

Tùy T1/T2, nhưng critical design SHOULD đạt:

- requirements traced;
- architecture model cập nhật;
- required ADR accepted;
- contracts validated;
- threat/security review hoàn thành;
- NFR support được chứng minh hoặc explicitly accepted risk;
- migration/rollback defined;
- observability defined;
- task decomposition linked;
- verification gates pass.

22.3 22.3 Implementation Done

- code/IaC/config merged;
- tests pass;
- traceability intact;
- contract/schema compatibility checks pass;
- deployment/rollout evidence captured khi cần;
- runtime verification hoặc follow-up condition được định nghĩa.

23 23. Metrics và Factory evaluation

Không đo bằng số trang tài liệu hoặc số Agent calls.

Metrics nên tập trung vào value và system health:

- idea-to-design lead time;
- design-to-implementation lead time;
- rework rate;
- escaped design defects;
- architecture drift count/age;
- untraced material changes;
- human review effort;
- Agent false-positive rate;
- Agent override rate;
- change failure rate;
- time-to-detect drift;
- cost per accepted change;
- debt age/concentration;
- policy exception age.

Trend quan trọng hơn một “architecture score” tổng hợp mơ hồ.

24 24. Agent evaluation harness

Agent cũng phải qua CI.

Ví dụ eval cases:

- detect direct DB coupling;
- detect missing timeout/retry semantics;
- không invent requirement;
- identify ADR conflict;
- preserve backward compatibility;
- classify trace level đúng;
- không expose Restricted context cho disallowed runtime;
- output conform schema.

Khi đổi agent prompt/model/runtime:

Candidate → Eval Suite → Shadow/Comparison → Limited Rollout → Promote/Rollback

Điều này là chìa khóa để thay Codex trong tương lai mà không dựa vào cảm giác.

25 25. Golden paths và escape hatches

25.1 25.1 Golden path

80% work nên đi theo:

Idea/Signal → Frame → Design → Verify → Approve → Implement → Observe → Learn

25.2 25.2 Escape hatches

Cần flow có kiểm soát cho:

- emergency production fix;
- security incident;
- research spike/prototype;
- legacy migration;
- regulatory deadline;
- tooling-only maintenance.

Escape hatch **MUST** không xóa traceability; nó có thể **defer** một số artifacts/gates với deadline backfill rõ ràng.

26 26. Maturity model

Level	Tên	Đặc điểm
0	Manual	docs + code + human review
1	AI Assisted	Codex hỗ trợ design/code/review
2	Structured	canonical specs + ADR + schemas + traceability
3	Agentic	orchestrator + specialist + critics
4	Governed	policies + permissions + evals + fitness functions
5	Living	Intent ↔ Reality ↔ Evolution Graph
6	Adaptive	runtime signals + meta-PDCA cải tiến Factory

Không nên nhảy thẳng Level 5/6 trước khi Level 2 traceability và canonicalization ổn định.

27 27. Lộ trình triển khai đề xuất

27.1 Phase 0 - 1 đến 2 tuần: Foundation

Mục tiêu: tạo canonical skeleton.

Deliverables:

- constitution v0;
- artifact IDs/status conventions;
- traceability policy T0/T1/T2;
- repo structure;
- base schemas;

- AGENTS.md generated/curated;
- CI skeleton;
- 5-10 evaluation cases đầu tiên.

Success criteria:

- một change nhỏ đi qua end-to-end flow;
- có bidirectional trace từ task đến requirement/test;
- không cần spreadsheet/manual registry bên ngoài Git.

27.2 Phase 1 - 0 đến 3 tháng: Prove the process

Dùng:

- VS Code;
- Codex;
- Git;
- structured design files;
- Graphify;
- scripts;
-

Chỉ cần bốn logical agents:

- Orchestrator
- Designer
- Reviewer
- Publisher/Executor

Mục tiêu: tìm ra artifact set tối thiểu và chất lượng traceability thực sự cần thiết.

27.3 Phase 2 - 3 đến 12 tháng: Governed agentic flow

Bổ sung:

- MCP tool layer;
- explicit Intent Graph;
- Reality Graph/Graphify integration;
- Traceability service/tool;
- impact analysis;
- specialist Architecture/Security/Data/Reliability Agents;
- fitness functions;
- agent eval pipeline;
- Factory metrics.

27.4 Phase 3 - 1 đến 3 năm: Factory Control Plane

Tách khỏi workstation:

- Agent Runtime Interface + adapters;
- runtime router;
- policy engine;
- shared graph services;

- audit/observability;
- cross-repo evolution graph;
- organization-level ownership and standards;
- VS Code, CI, web và other IDE là clients.

28. Minimum Viable SDF: phạm vi nên xây trước

Để giữ Lean, MVP không nên xây toàn bộ tầm nhìn.

MVP MUST có:

1. canonical artifact structure;
2. IDs + lifecycle/status;
3. traceability T1/T2;
4. Orchestrator/Designer/Reviewer roles trên Codex;
5. AGENTS.md/project config;
6. architecture/traceability lint;
7. Graphify Reality Graph thử nghiệm;
8. PR/CI gate;
9. basic eval suite;
10. publication pipeline Markdown → DOCX/HTML/PDF nếu cần.

MVP SHOULD chưa xây:

- custom distributed agent scheduler;
- complex vector/memory platform;
- autonomous production deploy;
- universal provider router;
- enterprise graph DB nếu repo-scale graph đủ dùng;
- quá nhiều specialist agents.

29. Acceptance criteria cho SDF v1

Một SDF v1 được xem là đạt khi có thể chứng minh end-to-end với một real change:

1. Idea/problem được capture có ID.
2. Requirement/NFR có acceptance criteria.
3. Design model và ADR cần thiết được tạo.
4. Design review phát hiện được một seeded inconsistency trong eval.
5. Work items được sinh và trace ngược lên design.
6. Developer/Codex thực hiện task trong VS Code/CLI.
7. PR diff tự xác định affected components/requirements ở mức chấp nhận được.
8. CI bắt được orphan T1/T2 change.
9. Tests/evidence đóng trace chain.
10. Reality Graph cập nhật sau merge.
11. Một intentional drift test tạo finding.
12. Tất cả derived outputs có thể rebuild từ clean checkout.
13. Khi tắt Graphify/AI, team vẫn build/test/review được ở degraded manual mode.

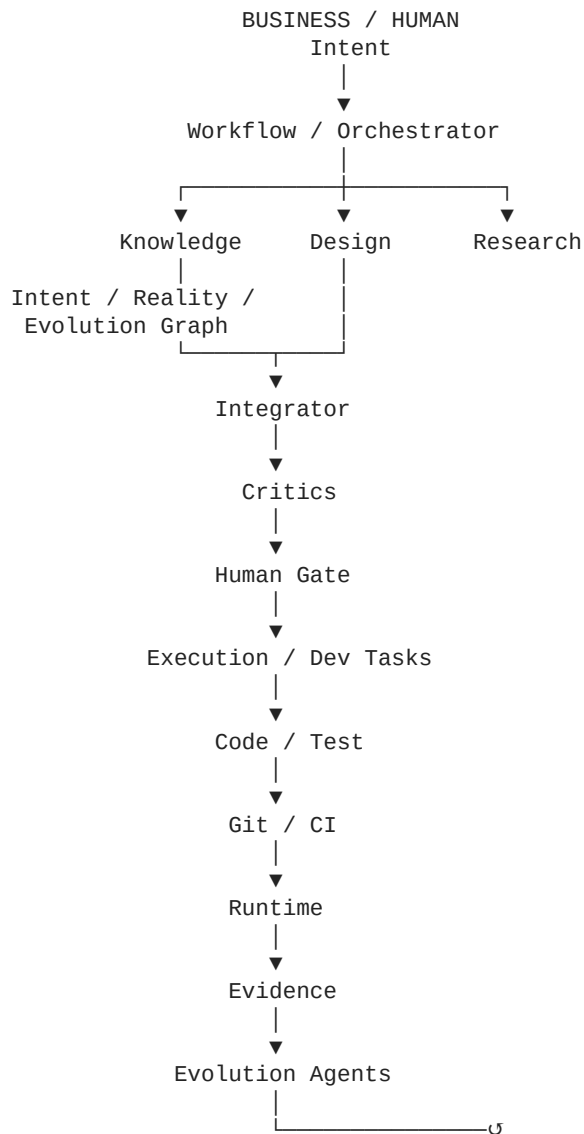
14. Ít nhất một Reviewer Agent chạy được qua runtime adapter/eval boundary mà không thay canonical artifacts.

30 30. Decision log ban đầu

Các quyết định baseline trước khi triển khai:

ID	Decision	Rationale
SDF-DEC-001	Git + canonical structured artifacts là source of truth	audit, diff, version, portability
SDF-DEC-002	Knowledge Graph là derived/query layer	tránh graph inference trở thành authority
SDF-DEC-003	Traceability many-to-many, bidirectional, risk-based	đủ rigor nhưng hạn chế bureaucracy
SDF-DEC-004	Lean + PDCA + continuous flow là operating baseline	process thích nghi, không khóa Scrum
SDF-DEC-005	VS Code + Codex là bootstrap stack	feedback loop ngắn, ít platform work
SDF-DEC-006	Core logic không nằm trong extension	long-term portability
SDF-DEC-007	MCP cho tools/context; ARI cho Agent Runtime	separation of concerns
SDF-DEC-008	Codex-first, not Codex-dependent	tận dụng hiện tại nhưng giữ exit strategy
SDF-DEC-009	Human gate cho irreversible/high-risk decisions	accountability
SDF-DEC-010	Agent/eval/policy đều versioned	reproducibility và safe evolution

31 31. Kiến trúc mục tiêu tóm tắt



CONTROL PLANE ACROSS ALL LAYERS
Constitution • Policy • Permission • Fitness • Eval • Audit

32 32. Kết luận

AI-Native SDF nên được hiểu là **hệ điều hành cho engineering intent**, không phải công cụ viết design docs.

Lỗi bền vững là:

Intent → Decision → Design → Task → Implementation → Verification → Runtime Evidence → Learning → Evolution

Các công cụ có thể thay đổi:

- VS Code có thể được thay bằng IDE khác;
- Codex có thể được thay bằng Agent Runtime khác;

- Graphify có thể được thay bằng Reality Graph adapter khác;
- Scrum có thể dùng hoặc không;
- model AI có thể đổi theo quality/cost/security.

Nhưng bốn invariants không đổi:

1. canonical truth có version;
2. material changes có traceability;
3. derived knowledge có provenance và rebuild được;
4. human giữ accountability đối với irreversible/high-risk decisions.

Với baseline này, bước tiếp theo hợp lý không phải xây một platform lớn, mà là **chọn một real software change và dùng nó làm “walking skeleton” để xây SDF Phase 0/1 theo đúng chính process mà Factory sẽ quản lý.**

33 Phụ lục A. Artifact ID conventions gợi ý

Prefix	Artifact
PROB	Problem
OUT	Outcome
FR	Functional Requirement
NFR	Non-functional Requirement
CMP	Architecture Component
ADR	Architecture Decision Record
API	API Contract
EVT	Event Contract
DATA	Data Model/Contract
THR	Threat
MIT	Mitigation
CP	Change Proposal
DEV	Development Task
TEST	Test/Verification
INC	Incident
DEBT	Technical Debt
DRIFT	Design Drift
FIND	Review Finding
POL	Policy
FIT	Fitness Function

34 Phụ lục B. Ví dụ traceability record

```
id: DEV-142
title: "Add asynchronous document indexing"
trace_level: T2
implements:
  - FR-023
supports_nfr:
  - NFR-011
changes:
  - CMP-INDEXER
governed_by:
  - ADR-012
contracts:
  - EVT-DOC-INDEX-001
security:
  - THR-031
verified_by:
  - TEST-INT-044
  - TEST-LOAD-012
runtime_evidence:
  - SLO-SEARCH-002
status: verified
```

35 Phụ lục C. Ví dụ canonical Agent Spec

```
id: architecture-reviewer
version: 1.0.0
purpose: "Detect design inconsistency and policy violations"
required_capabilities:
  - structured_output
  - repository_read
  - mcp
inputs:
  - requirements
  - architecture
  - decisions
  - change_diff
tools:
  - intent_graph.query
  - reality_graph.query
  - repository.read
permissions:
  write_repository: false
  approve_architecture: false
output_schema: design-finding.v2
```

36 Phụ lục D. Các nguồn kỹ thuật hiện hành tham khảo

Truy cập ngày 19/09/2026.

1. OpenAI Codex - Custom instructions with AGENTS.md: <https://developers.openai.com/codex/agent-configuration/agents-md>
2. OpenAI Codex - Subagents: <https://developers.openai.com/codex/subagents>
3. OpenAI Codex - Configuration Reference: <https://developers.openai.com/codex/config-reference>
4. OpenAI Codex - Model Context Protocol: <https://developers.openai.com/codex/mcp>
5. OpenAI Codex - CLI: <https://developers.openai.com/codex/cli>
6. OpenAI Codex - MCP server removal / app server note: <https://developers.openai.com/codex/mcp-server>

7. Visual Studio Code - AI/agent overview and MCP positioning: <https://code.visualstudio.com/>
8. Graphify Labs - Graphify open-source repository: <https://github.com/Graphify-Labs/graphify>